

PROGRAMMING FOR PROBLEM SOLVING

UNIT 1 C Fundamentals & Decision Making

PREMANAND S

Assistant Professor

Department of Computational Intelligence

SRM Institute of Science and Technology

Kattankulathur Campus

First Year Engineering

Operator Precedence

1 WHY DO WE NEED OPERATOR PRECEDENCE?



When an expression has **more than one operator**, C must decide which operator to evaluate first. This decision is made using **operator precedence**.

EXAMPLE

Consider the expression:

```
result = 10 + 5 * 2;
```

If we evaluate from **LEFT to RIGHT**

$10 + 5 = 15$

$15 * 2 = 30$

result = 30

✗ Wrong

BUT

What C actually does

$5 * 2 = 10$

$10 + 10 = 20$

result = 20

✓ Correct

DEFINITION

Operator precedence tells C **which operator should be evaluated first** when an expression contains multiple operators.

KEY IDEA

Higher precedence → **evaluated earlier**

VISUALIZE THE PRIORITY

$10 + 5 * 2$



First evaluate ***** (multiplication)
then evaluate **+** (addition)



TAKEAWAY

C does not always evaluate from left to right. It follows **operator precedence** to get the correct result.

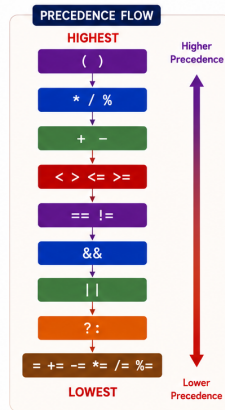
Remember:

Parentheses () have the highest precedence. They can override the normal order.

Operator Precedence

ORDER OF EVALUATION (HIGHEST TO LOWEST)

Priority	Operators	Meaning / Category	Example
1	()	Parentheses Expressions inside parentheses are evaluated first.	$(a + b) * c$
2	* / %	Multiplication / Division / Modulus Multiplication, Division and Modulus.	$a * b / c$
3	+ -	Addition / Subtraction Addition and Subtraction.	$a + b - c$
4	< > <= >=	Relational Operators Less than, Greater than, Less than or equal to, Greater than or equal to.	$a < b$
5	== !=	Equality Operators Equal to, Not equal to.	$a == b$
6	&&	Logical AND Logical AND.	$a \&\& b$
7		Logical OR Logical OR.	$a b$
8	? :	Conditional Operator Ternary conditional operator.	$a > b ? a : b$
9	= += -= *= /= %=	Assignment Operators Assignment and compound assignment operators.	$a = b + c$



Operator Precedence - Example



Follow the operator precedence order. If operators have the same precedence, evaluate from left to right.

1 Example 1 (Simple)

Expression: $10 + 5 * 2$

Steps:

- 1 $*$ has higher precedence than $+$ $10 + (5 * 2)$
- 2 Evaluate $*$ $10 + 10$
- 3 Now evaluate $+$ 20 Result = 20

2 Example 2 (With Parentheses)

Expression: $(10 + 5) * 2$

Steps:

- 1 Parentheses first $(10 + 5) * 2$
- 2 Evaluate inside $()$ $15 * 2$
- 3 Now evaluate $*$ 30 Result = 30

3 Example 3 (Medium)

Expression: $20 - 8 / 2 + 3 * 4$

Steps:

- 1 $/$ and $*$ have higher precedence $20 - (8 / 2) + (3 * 4)$
- 2 Evaluate $/$ and $*$ $20 - 4 + 12$
- 3 $+$ and $-$ have same precedence $\rightarrow$ left to right
- 4 Evaluate from left to right $(20 - 4) + 12 \rightarrow 16 + 12$
- 5 Final result 28 Result = 28

4 Example 4 (Better Challenge)

Expression: $10 + 6 * 2 - 8 / 4$

Steps:

- 1 $*$ and $/$ have higher precedence $10 + (6 * 2) - (8 / 4)$
- 2 Evaluate $*$ and $/$ $10 + 12 - 2$
- 3 $+$ and $-$ have same precedence $\rightarrow$ left to right
- 4 Evaluate from left to right $(10 + 12) - 2 \rightarrow 22 - 2$
- 5 Final result 20 Result = 20

Associativity



When operators have the **SAME PRECEDENCE**, **associativity** decides the order of evaluation.
C uses **Left to Right** associativity for most operators.

ASSOCIATIVITY OF COMMON OPERATORS

Operators	Associativity
+ - * / %	Left to Right
< > <= >=	Left to Right
== !=	Left to Right
&&	Left to Right
	Left to Right
= += -= *= /= %=	Right to Left
? :	Right to Left

EXAMPLE 1: SAME PRECEDENCE (+ and -)

Expression: $20 - 5 + 2$

Step-by-step evaluation (Left to Right):

- Both - and + have same precedence.
- Evaluate from left to right.
- $(20 - 5) + 2$
- $15 + 2$
- 17

$20 - 5 + 2$
 $(20 - 5) + 2$
 $15 + 2$
17

Result = 17

Observation

- Even though + comes after -, we still evaluate from left to right because they have the same precedence.

EXAMPLE 2: SAME PRECEDENCE (* and /)

Expression: $100 / 5 * 2$

Step-by-step evaluation (Left to Right):

- Both / and * have same precedence.
- Evaluate from left to right.
- $(100 / 5) * 2$
- $20 * 2$
- 40

Result = 40

EXAMPLE 3: ASSIGNMENT (Right to Left)

Code: $a = b = c = 10$

Step-by-step evaluation (Right to Left):

- Assignment operators are Right to Left.
- First: $c = 10$
- Then: $b = c \rightarrow b = 10$
- Finally: $a = b \rightarrow a = 10$

Final Values

$a = 10$
 $b = 10$
 $c = 10$

KEY TAKEAWAYS

- Precedence decides which operator goes first.
- When precedence is same, **associativity** decides the order.
- Most operators in C are evaluated from **Left to Right**.
- Assignment ($=$, $+=$, $-=$, etc.) and Conditional ($? :$) are **Right to Left**.



TIP

Parentheses override both precedence and associativity.



Always use **parentheses** when in doubt. It makes your intention clear and your program bug-free!

Examples to Understand



Let's evaluate some expressions. Follow **precedence** first.

If operators have the same precedence, evaluate from **Left to Right** (except **assignment** and **? :**).

EXAMPLE 1

Expression:

$50 - 20 / 4 + 3 * 5$

Step-by-step evaluation:

- 1 / and * have higher precedence
 - 2 Evaluate / and *
 - 3 + and - have same precedence
→ evaluate from Left to Right
 - 4 $50 - 5$
 - 5 $45 + 15$
- $50 - (20 / 4) + (3 * 5)$
 $50 - 5 + 15$
 $(50 - 5) + 15$
 $45 + 15$
60

Result = 60

EXAMPLE 2

Expression:

$(2 + 3) * (8 - 4) / 2$

Step-by-step evaluation:

- 1 Parentheses first
 - 2 Evaluate inside ()
 - 3 * and / have same precedence
→ evaluate from Left to Right
 - 4 $5 * 4$
 - 5 $20 / 2$
- $(5) * (4) / 2$
 $5 * 4 / 2$
 $(5 * 4) / 2$
 $20 / 2$
10

Result = 10

EXAMPLE 3

Expression:

$10 + 20 > 25 \&\& 5 < 10$

Step-by-step evaluation:

- 1 + has higher precedence than >
 - 2 Evaluate +
 - 3 < and > have higher precedence than &&
 - 4 Evaluate > and <
 - 5 Evaluate &&
- $(10 + 20) > 25 \&\& 5 < 10$
 $30 > 25 \&\& 5 < 10$
 $(30 > 25) \&\& (5 < 10)$
 $1 \&\& 1$
1

Result = 1 (True)

In C: True = 1, False = 0

EXAMPLE 4 (With Conditional Operator)

Expression:

$(10 > 5) ? 100 : 200$

Step-by-step evaluation:

- 1 Evaluate condition $(10 > 5)$
 - 2 Condition is True (non-zero)
→ choose first value
 - 3 200 is not evaluated
 - 4 Final result
- $(1) ? 100 : 200$
100
—
100

Result = 100

? : has lower precedence than most operators but higher than assignment (=).

PRECEDENCE QUICK REFERENCE

	()		Highest Priority
1	()		
2	* / %		
3	+ -		
4	< > <= >=		
5	== !=		
6	&&		
7	?:		
8	= += -= *= /= %=		
9			Lowest Priority

Operator Precedence — Questions to Solve

- ① Evaluate the following expression:

$$10 + 5 * 2$$

- ② Evaluate:

$$20 - 8/2 + 3 * 4$$

- ③ Evaluate:

$$(10 + 5) * 2$$

- ④ Evaluate:

$$50 - 20/4 + 3 * 5$$

- ⑤ Evaluate:

$$100/5 * 2 + 3 - 6$$

Operator Precedence — Questions to Solve

- 1 Predict the output:

$5 > 3 \ \&\& \ 10 < 20$

- 2 Predict the output:

$5 > 3 \ || \ 2 > 10$

- 3 Predict the output:

$!(5 > 3) \ \&\& \ (2 < 4)$

- 4 Evaluate:

$10 + 20 > 25 \ \&\& \ 5 < 10$

Operator Precedence — Challenge

- ① Without using a calculator, find the result:

$$8 + 6 * 2 - 4/2$$

- ② Find the result:

$$(8 + 6) * (10 - 4)/2$$

- ③ Find the result:

$$25 - 6 * 2 + 12/3$$

- ④ Write the order in which the operators are evaluated:

$$10 + 4 * 3 - 8/2$$

- ⑤ Explain why the following two expressions produce different results:

$$10 + 5 * 2 \quad \text{and} \quad (10 + 5) * 2$$

Expressions



What is an Expression?

An expression is a combination of **values**, **variables**, and **operators** that produces a result.

Think of it like a recipe:



Simple Expressions

`a + b`

`x * 10`

`marks >= 50`

`a = b + 5`

Different Types of Expressions

● Arithmetic Expression:

`a + b * c`

● Relational Expression:

`a > b`

● Logical Expression:

`a > 0 && b > 0`

● Assignment Expression:

`x = 10`



Key Idea

An expression is evaluated to produce a **value**.



Notes:

- Expressions can be used in assignments, conditions, loops, function calls, and more.
- C evaluates expressions using operator **precedence** and **associativity**.

Example:

```
int x = a + b * c - 5;  
// x gets the final result
```

Decision Making

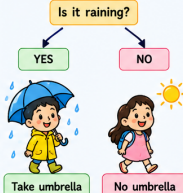


1 WHAT IS DECISION MAKING?

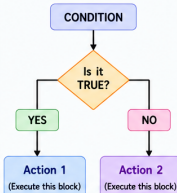
A program often needs to **choose between different actions** depending on a **condition**.

"A condition produces a result (**TRUE** or **FALSE**) and the program decides what to do based on that result."

2 REAL-LIFE EXAMPLE



3 IN PROGRAMMING



4 SIMPLE C EXAMPLE

```
#include <stdio.h>
int main() {
    int age = 20;

    if (age >= 18)
        printf("Eligible to vote\n");
    return 0;
}
```

Here:

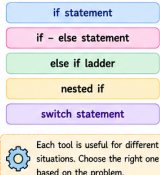


5 WHY DO WE NEED DECISION MAKING?

- ✓ Check whether a number is positive or negative
- ✓ Check pass or fail
- ✓ Check eligibility
- ✓ Find the largest number
- ✓ Validate user input
- ✓ Apply discounts
- ✓ Control program flow
- ✓ Solve real-world problems



6 DECISION-MAKING TOOLS IN C



KEY TAKEAWAYS

- 1 Decision making helps the program choose between different actions.
- 2 Conditions evaluate to **TRUE** or **FALSE**.
- 3 C provides **if**, **if-else**, **else if**, **nested if** and **switch** for decision making.
- 4 Decision making is essential to solve real-world problems.

Think
Evaluate
Decide
Code
Create!

Better
Programs
Brighter
Futures



Programs don't just calculate. They also **think**!



Small Decisions → Big Impact → Real Solutions

Decision Making - Types



C provides different **decision-making statements** to choose the right action based on a condition.

"Same problem,
Different situations,
Different decisions!"



1 SIMPLE if

Used when we want to execute an action **only when a condition is true**.

```
if (age >= 18)
    printf("Eligible");
```

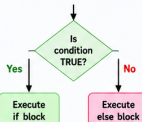


Use when there is **only one condition**.

2 if - else

Used when there are **two possible paths** (true or false).

```
if (marks >= 40)
    printf("Pass");
else
    printf("Fail");
```

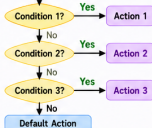


Use when there are **two alternatives**.

3 else if Ladder

Used when we need to check **multiple conditions** one after another.

```
if (marks >= 90)
    printf("A");
else if (marks >= 75)
    printf("B");
else if (marks >= 60)
    printf("C");
else
    printf("D");
```

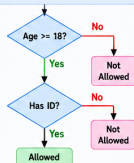


Use when there are **multiple choices or ranges**.

4 Nested if

An **if statement inside another if statement**.

```
if (age >= 18) {
    if (hasID == 1)
        printf("Allowed");
}
```

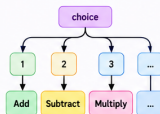


Use when a decision **depends on another decision**.

5 switch

Used when we need to choose among **multiple fixed choices** (e.g., menu, options).

```
switch(choice) {
    case 1: printf("Add");
            break;
    case 2: printf("Subtract");
            break;
    default: printf("Invalid");
}
```



Use when there are **multiple fixed options (discrete values)**.

QUICK COMPARISON

Type	Best Suited For
if	One condition
if - else	Two alternatives (true or false)
else if ladder	Multiple conditions
Nested if	Decision inside another decision
switch	Multiple fixed choices (menu/options)

★ KEY TAKEAWAYS

- ✓ Decision making helps the program choose the right action.
- ✓ Conditions evaluate to **TRUE** or **FALSE**.
- ✓ C provides **if, if-else, else if ladder, nested if** and **switch**.
- ✓ Each type is useful in different situations.
- ★ Choose the right tool based on your problem.



REAL-WORLD ANALOGY

- **if** → Like checking if it is raining.
- **if - else** → Like pass or fail.
- **else if** → Like grading (A, B, C, D).
- **nested if** → Like checking age and ID.
- **switch** → Like a vending machine (choose an option).



Good programs make **good decisions!**



Think → Decide → Code → Create a Better Tomorrow!

Simple IF Statement

1 WHAT IS if ?

The **if** statement tells C to execute a statement **only** when the condition is true.

"If a condition is true, do this!"

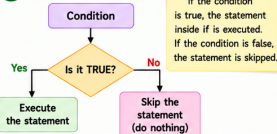


2 SYNTAX

```
if (condition)
{
    statement;
}
```

Statement can be a single statement or a block of statements.

3 HOW IT WORKS?



4 EXAMPLE 1

Check whether a person is eligible to vote.

```
#include <stdio.h>
int main() {
    int age = 20;

    if (age >= 18) {
        printf("You are eligible");
    }
    return 0;
}
```

5 HOW IT GETS EVALUATED?

Here, age = 20

age >= 18
20 >= 18

TRUE

Statement executed
Output:
You are eligible

6 EXAMPLE 2

When the condition is false.

```
#include <stdio.h>
int main() {
    int age = 15;

    if (age >= 18) {
        printf("You are eligible");
    }
    return 0;
}
```

7 EVALUATION

Here, age = 15

age >= 18
15 >= 18

FALSE

Statement is skipped
No output

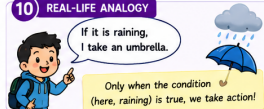
8 IMPORTANT POINTS

- 1 The statement inside **if** is executed **only** when the condition is true.
- 2 If the condition is false, the **statement is skipped**.
- 3 There is **no else part** in a simple if.
- 4 The condition usually involves relational or logical expressions.

9 QUICK SUMMARY

Condition Result	Action
TRUE	Execute the statement inside if
FALSE	Do nothing (skip the statement)

10 REAL-LIFE ANALOGY



Write conditions. Build decisions. Create smarter programs!

Learn → Practice → Solve → Grow

Simple IF Statement and Relational Operators

1 RECAP: if STATEMENT

```
if (condition)
{
    statement;
}
```

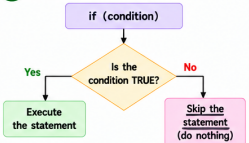
The statement is executed only when the condition is true.



2 RELATIONAL OPERATORS IN C

Operator	Meaning	Example	Result (if a = 5, b = 3)
>	Greater than	a > b	1 (true)
<	Less than	a < b	0 (false)
>=	Greater than or equal to	a >= b	1 (true)
<=	Less than or equal to	a <= b	0 (false)
==	Equal to	a == b	0 (false)
!=	Not equal to	a != b	1 (true)

3 HOW IT WORKS?



4 EXAMPLES USING RELATIONAL OPERATORS

1 Greater than (>)

```
#include <stdio.h>
int main() {
    int num = 10;
    if (num > 0) {
        printf("Positive");
    }
    return 0;
}
```

Output (num = 10)
Positive

Checks if the number is greater than 0.

2 Less than (<)

```
#include <stdio.h>
int main() {
    int num = -5;
    if (num < 0) {
        printf("Negative");
    }
    return 0;
}
```

Output (num = -5)
Negative

Checks if the number is less than 0.

3 Greater than or equal (>=)

```
#include <stdio.h>
int main() {
    int age = 18;
    if (age >= 18) {
        printf("Eligible");
    }
    return 0;
}
```

Output (age = 18)
Eligible

Checks if age is 18 or more.

4 Less than or equal (<=)

```
#include <stdio.h>
int main() {
    int marks = 40;
    if (marks <= 40) {
        printf("Need to improve");
    }
    return 0;
}
```

Output (marks = 40)
Need to improve

Checks if marks is 40 or less.

5 Equal to (==)

```
#include <stdio.h>
int main() {
    int ch = 1;
    if (ch == 1) {
        printf("You chose 1");
    }
    return 0;
}
```

Output (ch = 1)
You chose 1

Checks if two values are equal.

6 Not equal to (!=)

```
#include <stdio.h>
int main() {
    int x = 5;
    if (x != 0) {
        printf("Non-zero");
    }
    return 0;
}
```

Output (x = 5)
Non-zero

Checks if two values are not equal.

5 IMPORTANT POINTS

- ✓ Relational operators are used in conditions.
- ✓ They produce a result: 1 (true) or 0 (false).
- ✓ Used with the if statement to control the flow of the program.
- ✓ Commonly used to compare numbers, characters, or variables.

6 THINGS TO REMEMBER

- 1 Use == for comparison, not = (which is assignment).
- 2 Relational operators have lower precedence than arithmetic operators.
- 3 The result of a relational expression is an integer (1 or 0).
- 4 Use meaningful conditions to make your program clear.

7 REAL-LIFE EXAMPLES

- 👤 Check if a student passed (marks >= 40)
- 👤 Check if a person is eligible (age >= 18)
- 👤 Check if a number is positive, negative or zero
- ✅ Validate user input (e.g., choice between 1 and 5)

Compare using relational operators, make smarter decisions!



Conditions turn possibilities into actions!



Small Conditions → Big Impact → Better Programs!

Simple IF Statement and Logical Operators

1 WHY LOGICAL OPERATORS?

Sometimes we need to check **more than one condition** together.



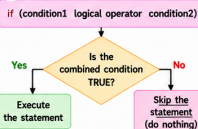
"Logical operators help us combine **multiple conditions** and make better decisions."



2 LOGICAL OPERATORS IN C

Operator	Meaning	Example	Result (if a = 1, b = 0)
&&	Logical AND (both conditions true)	a && b	0 (false)
	Logical OR (at least one true)	a b	1 (true)
!	Logical NOT (inverts the result)	!a !b	0 (false) 1 (true)

3 HOW IT WORKS?



Both conditions are evaluated using a logical operator and the result is either TRUE (1) or FALSE (0).

4 EXAMPLES USING LOGICAL OPERATORS

A Using && (Logical AND)

Both conditions must be true.

```
#include <stdio.h>
int main() {
    int age = 20;
    int hasID = 1;

    if (age >= 18 && hasID == 1) {
        printf("You are allowed to enter");
    }
    return 0;
}
```

Example:

```
age >= 18 → 1 (true)
hasID == 1 → 1 (true)
1 && 1 → 1 (true)
```

Output:

You are allowed to enter

Both age and ID are required.

B Using || (Logical OR)

At least one condition must be true.

```
#include <stdio.h>
int main() {
    int age = 17;
    int isStudent = 1;

    if (age >= 18 || isStudent == 1) {
        printf("You get a discount");
    }
    return 0;
}
```

Example:

```
age >= 18 → 0 (false)
isStudent == 1 → 1 (true)
0 || 1 → 1 (true)
```

Output:

You get a discount

Either age is 18 or the person is a student.

C Using ! (Logical NOT)

Inverts the result of a condition.

```
#include <stdio.h>
int main() {
    int isPresent = 0;

    if (!isPresent) {
        printf("You are absent");
    }
    return 0;
}
```

Example:

```
isPresent = 0
!isPresent → 1 (true)
```

Output:

You are absent

Executes when the condition is false.

5 REAL-WORLD EXAMPLES

- ✓ Check if a person is adult and has a valid ID.
- ✓ Allow login if username or email is correct.
- ✓ Check if a number is positive and even.
- ✓ Give discount if a person is a student or senior citizen.
- ✓ Validate input (e.g., age between 18 and 60).

Logical operators help us handle real-world situations!



6 KEY TAKEAWAYS

- 1 && (AND) → true only if both conditions are true.
- 2 || (OR) → true if at least one condition is true.
- 3 ! (NOT) → inverts the result.
- 4 Used with if to make better decisions.
- 5 Result is always 1 (true) or 0 (false).
- 6 Helps solve real-world problems.

7 COMMON MISTAKES

- ✗ Using & instead of && (bitwise vs logical).
- ✗ Using | instead of ||.
- ✗ Forgetting parentheses in complex conditions.
- ✗ Assuming non-zero values are false (they are true).
- ✗ Not understanding the difference between AND and OR.

Think Clearly
Write Correctly
Debug Happily! 🧐



Combine conditions. Build smarter programs!



Better Conditions → Better Decisions → Brighter Solutions!

Practice Problems — if Statements

- 1 Write a C program to check whether a given number is **positive**. Use an `if` statement.
- 2 Write a C program to check whether a given number is **even**. Use the modulus operator (%) and `if`.
- 3 Write a C program to check whether a student has **passed**. Assume the pass mark is 40.
- 4 Write a C program to check whether a person is **eligible to vote**. The minimum age is 18.
- 5 Write a C program to check whether a given number is **between 10 and 50**.

Practice Problems — if + Logical Operators

- ① Write a C program to check whether a number is **positive and even**.
- ② Write a C program to check whether a person is eligible for a discount if the person is **60 years or older OR is a student**.
- ③ Write a C program to check whether a student is eligible for an exam if:
 - attendance is at least 75%, **AND**
 - the student has paid the exam fee.
- ④ Write a C program to check whether a given number is **positive, even, and less than 100**.
- ⑤ Write a C program to check whether a person is allowed to enter a competition if their age is between **18 and 30**.

Challenge — Think Before You Code!

Problem

Write a C program to check whether a given number satisfies **all** of the following conditions:

- The number is positive.
- The number is even.
- The number is greater than 10.
- The number is less than 100.

Hint

Can you combine all four conditions using the **logical AND (&&)** operator?

Practice Problem 1 — Check Positive Number

C Program

```
#include <stdio.h>
int main()
{
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);

    if (number > 0)
    {
        printf("The number is positive");
    }
    return 0;
}
```

Practice Problem 2 — Check Even Number

C Program

```
#include <stdio.h>
int main()
{
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);

    if (number % 2 == 0)
    {
        printf("The number is even");
    }
    return 0;
}
```

Practice Problem 3 — Pass Eligibility

C Program

```
#include <stdio.h>
int main()
{
    int marks;
    printf("Enter your marks: ");
    scanf("%d", &marks);

    if (marks >= 40)
    {
        printf("The student has passed");
    }
    return 0;
}
```

Practice Problem 4 — Voting Eligibility

C Program

```
#include <stdio.h>
int main()
{
    int age;
    printf("Enter your age: ");
    scanf("%d", &age);

    if (age >= 18)
    {
        printf("Eligible to vote");
    }
    return 0;
}
```

Practice Problem 5 — Number Between 10 and 50

C Program

```
#include <stdio.h>
int main()
{
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if (number >= 10 && number <= 50)
    {
        printf("The number is between 10 and 50");
    }
    return 0;
}
```

Practice Problem 6 — Positive Number with Logical AND

C Program

```
#include <stdio.h>
int main()
{
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);

    if (number > 0 && number % 2 == 0)
    {
        printf("The number is positive and even");
    }
    return 0;
}
```

Practice Problem 7 — Discount Eligibility using —

C Program

```
#include <stdio.h>
int main()
{
    int age, student;
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("Are you a student? (1-Yes, 0-No): ");
    scanf("%d", &student);

    if (age >= 60 || student == 1)
    {
        printf("Eligible for discount");
    }
    return 0;
}
```

Practice Problem 8 — Exam Eligibility using AND

C Program

```
#include <stdio.h>
int main()
{
    float attendance;
    int feePaid;
    printf("Enter attendance percentage: ");
    scanf("%f", &attendance);
    printf("Has the exam fee been paid? (1-Yes, 0-No): ");
    scanf("%d", &feePaid);
    if (attendance >= 75 && feePaid == 1)
    {
        printf("Eligible to write the examination");
    }
    return 0;
}
```

Practice Problem 9 — Multiple Conditions

C Program

```
#include <stdio.h>
int main()
{
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);
    if (number > 0 && number % 2 == 0 && number < 100)
    {
        printf("The number is positive, even, and less than 100")
    }
    return 0;
}
```

Practice Problem 10 — Age Between 18 and 30

C Program

```
#include <stdio.h>
int main()
{
    int age;
    printf("Enter your age: ");
    scanf("%d", &age);
    if (age >= 18 && age <= 30)
    {
        printf("Eligible to enter the competition");
    }
    return 0;
}
```

Challenge Problem — Think Before You Code!

C Program

```
#include <stdio.h>
int main()
{
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);
    if (number > 0 &&
        number % 2 == 0 &&
        number > 10 &&
        number < 100)
    {
        printf("The number satisfies all conditions");
    }
    return 0;
}
```

Practice Questions — if Statements

- 1 Write a C program to check whether a given number is **greater than 100**.
- 2 Write a C program to check whether a given number is **divisible by 5**.
- 3 Write a C program to check whether a given number is **positive and odd**.
- 4 Write a C program to check whether a student's mark is **between 50 and 100**.
- 5 Write a C program to check whether a person is eligible for a senior citizen benefit if the age is **60 or above**.

Practice Questions — Logical Operators

- ① Write a C program to check whether a number is **divisible by both 3 and 5**.
- ② Write a C program to check whether a number is **between 1 and 100 and divisible by 10**.
- ③ Write a C program to check whether a student is eligible for a scholarship if:
 - marks are at least 85, and
 - attendance is at least 75%.
- ④ Write a C program to check whether a person can enter a movie if the age is **18 or above OR the person is accompanied by a parent**.
- ⑤ Write a C program to check whether a given number is **positive, odd, and less than 50**.

Predict the Output — if

- ❶ What will be the output?

```
int x = 15;  
if (x > 10)  
printf("A");  
if (x % 2 == 0)  
printf("B");
```

- ❷ What will be the output?

```
int a = 10, b = 20;  
if (a < b && b > 15)  
printf("Yes");
```

- ❸ What will be the output?

```
int n = 25;  
if (n > 0 && n < 20)  
printf("Valid");
```

Mini Challenge — Think Before You Code!

Problem

Write a C program to check whether a given three-digit number is **positive, even, and divisible by 3**.

Hint

Think about the three conditions separately first. Then combine them using the appropriate logical operator.